



Sri Eshwar
College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution
Affiliated to Anna University, Chennai



U23CB593 SYSTEM DESIGN LABORATORY

**DEPARTMENT OF COMPUTER AND COMMUNICATION
ENGINEERING**

**SRI ESHWAR COLLEGE OF ENGINEERING
KINATHUKADAVU COIMBATORE - 641202**



Sri Eshwar
College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution
Affiliated to Anna University, Chennai



DEPARTMENT OF COMPUTER AND COMMUNICATION ENGINEERING

BONAFIDE CERTIFICATE

Certified that this is the Bonafide record of work done by

Name: Mr. /Ms.

Register No: of 3rd Year

B.E/B.Tech in the

U23CB593 SYSTEM DESIGN LABORATORY during the 6th **Semester** of the academic year

2025 – 2026 (Even Semester).

Signature of faculty In-charge

Head of the department

Submitted for the practical examinations of Anna University, held on

Internal Examiner

External Examiner

TABLE OF CONTENT

EX. NO.	NAME OF THE EXPERIMENTS	PAGE NO	MARK (75)	SIGNATURE
1	DRAW LAYERED ARCHITECTURE DESIGN FOR A NOTE-TAKING APPLICATION			
2	PREPARE HIGH LEVEL DESIGN (HLD) AND LOW LEVEL DESIGN (LLD) FOR A URL SHORTENING SERVICE (TINY URL)			
3	THE WORKING OF REDIS CACHE FOR AN API BY EXPLAINING IN-MEMORY CACHING WITH TTL AND LRU EVICTION POLICY			
4	PRODUCER-CONSUMER SYSTEM DESIGN USING KAFKA TO DEMONSTRATE AN ORDER PROCESSING QUEUE			
5	DESIGN AND COMPARE SQL AND NOSQL SCHEMA FOR A BLOGGING PLATFORM, HIGHLIGHTING THE CONCEPTS OF NORMALIZATION AND DENORMALIZATION			
6	IMPLEMENTATION OF CIRCUIT BREAKER AND RETRY LOGIC USING RESILIENCE4J			
7	WORKING OF TOKEN BUCKET RATE LIMITER			
8	FAILOVER AND HEARTBEAT MECHANISMS IN A LOAD-BALANCED APPLICATION USING MOCK SERVICES			
9	DESIGN OF MICROSERVICES FOR E- COMMERCE CHECKOUT SYSTEM USING REST API			
10	IMPLEMENTATION OF PUB/SUB NOTIFICATION SERVICE USING RABBITMQ			

11	DEMONSTRATION OF API GATEWAY ARCHITECTURE USING EXPRESS.JS OR NGINX			
12	ARCHITECTURE DIAGRAM FOR REAL-TIME CHAT APPLICATION			
13	IMPLEMENTATION OF SERVLET-BASED USER MANAGEMENT WITH CRUD OPERATIONS.			
14	REST API ARCHITECTURE USING SPRING BOOT OR JAX-RS FOR A TASK MANAGEMENT SYSTEM.			
15	PREPARE A CAPSTONE SYSTEM DESIGN FOR A TWITTER-LIKE PLATFORM FOCUSING ON DATABASE SHARDING, FEED GENERATION, AND CACHING STRATEGIES			

Ex.no : 1	DRAW LAYERED ARCHITECTURE DESIGN FOR A NOTE-TAKING APPLICATION
Date :	

Aim

To design a layered architecture for a note-taking application using a component diagram and to explain the data flow between the client, server, and database.

System Overview

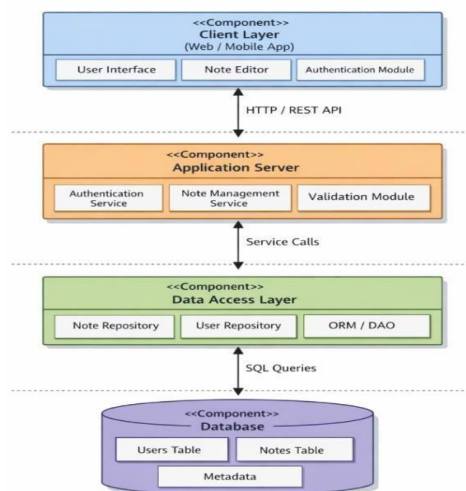
A note-taking application allows users to create, edit, view, and delete notes. The system follows a layered architecture to ensure modularity, scalability, and ease of maintenance.

Layered Architecture Components

The system is divided into the following layers:

1. Presentation Layer (Client)
2. Application / Business Logic Layer (Server)
3. Data Access Layer
4. Database Layer

Component Diagram



Data Flow Explanation

Scenario: Creating a New Note

1. User Action
 - User enters a note in the client application and clicks *Save*.
2. Client to Server
 - The client sends a POST request containing note details to the application server.
3. Server Processing
 - The server validates the user and note content.
 - Business logic processes the request.
4. Server to Data Access Layer
 - The request is passed to the data access layer to store the note.
5. Database Interaction
 - The note is saved in the database.
6. Response Flow
 - Success response is sent back from the database → server → client.
 - The client displays confirmation to the user.

Result

A layered architecture for a note-taking application was successfully designed using a component diagram, and the data flow between client, server, and database was clearly explained.

Ex.no : 2	PREPARE HIGH LEVEL DESIGN (HLD) AND LOW LEVEL DESIGN (LLD) FOR A URL SHORTENING SERVICE (TINY URL)
Date :	

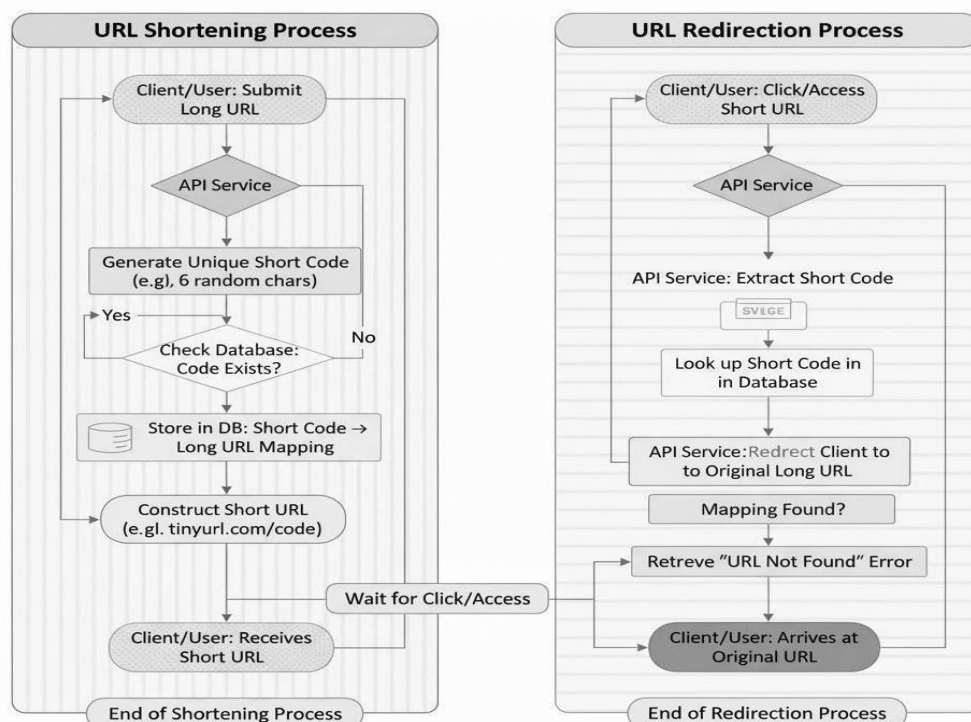
Aim

To Prepare High Level Design (HLD) and Low Level Design (LLD) for a URL Shortening service (TinyURL), including major components, database schema, and flow diagrams.

Procedure:

1. Identify system components: API, Database, and Redirect Service.
2. Define data flow for URL creation and redirection.
3. Design database schema to store short and long URLs.
4. Implement short code generation logic using Base62 encoding.
5. Test URL creation and redirection with sample inputs.

Flow diagram:



Low Level Design (LLD) :

Database Design :

Table: url_mapping		
Field Name	Data Type	Description
id	INT (PK)	Unique identifier
long_url	VARCHAR	Original URL
short_code	VARCHAR	Generated short code
created_at	TIMESTAMP	Creation time

Program:

```
import java.util.*;

import java.util.concurrent.ConcurrentHashMap;

public class URLShortener {

    private static final String BASE_URL =

    "https://tinyurl.com/"; private static final

    String CHARSET =

    "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";

    private static final Map<String, String> urlMap = new ConcurrentHashMap<>();

    public static String shortenURL(String longUrl) {

        String shortCode =

        generateCode();

        urlMap.put(shortCode, longUrl);

        return BASE_URL + shortCode;

    }

    public static String redirect(String shortCode) {

        return urlMap.getOrDefault(shortCode, "URL not found");

    }

}
```



```

    }

    private static String generateCode() {
        Random rand = new Random();
        StringBuilder code = new
        StringBuilder(); for (int i = 0; i < 6;
        i++)
            code.append(CHARSET.charAt(rand.nextInt(CHARSET.length())
            )); return code.toString();
    }
    public static void main(String[] args) {
        String longUrl = "https://example.com/some/long/path";
        String shortUrl = shortenURL(longUrl);
        System.out.println("Shortened: " + shortUrl);
        System.out.println("Redirected to: " +
        redirect(shortUrl.substring(BASE_URL.length())));
    }
}

```

Output :

Shortened: <https://tinyurl.com/aB9kLp>

Redirected to:

<https://example.com/some/long/path>

Result :

Thus the High Level Design (HLD) and Low Level Design (LLD) for a URL Shortening service (TinyURL), including major components, database schema, and flow diagrams was successfully designed.

Ex.no : 3	THE WORKING OF REDIS CACHE FOR AN API BY EXPLAINING IN-MEMORY CACHING WITH TTL AND LRU EVICTION POLICY
Date :	

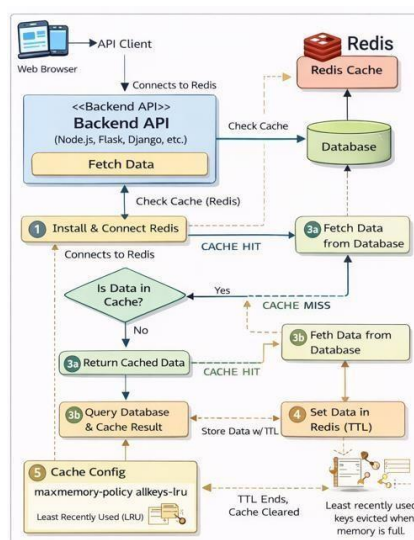
Aim:

To illustrate the working of Redis cache for an API by explaining in-memory caching with TTL (Time-To-Live) and LRU (Least Recently Used) eviction policy.

Procedure:

1. Install Redis and connect it to your backend API (e.g., Node.js, Python Flask, or Django).
2. The API endpoint fetches data (e.g., product details) from a database.
3. Integrate Redis Cache:
 - Before querying the database, the API checks if the data exists in Redis.
 - If present, the cached data is returned (cache hit).
 - If not, data is fetched from the database and stored in Redis with a TTL.
4. Configure Redis to use maxmemory-policy allkeys-lru so least recently used keys are deleted when memory is full.
5. After TTL expires, the cache is cleared automatically, forcing the next request to fetch from DB again.

Flow diagram: Redis Cache Integration with Backend API



Program:

```
import redis.clients.jedis.Jedis;

import

redis.clients.jedis.params.SetParams;

import java.time.Instant;

import java.util.Random;

import java.util.concurrent.TimeUnit;

public class RedisCacheDemo {

    // Simulated DB

    (slow) static class

    SlowDB {

        String fetch(String key) {

            try { TimeUnit.MILLISECONDS.sleep(200); } catch (InterruptedException

ignored) {}

            return "VALUE_FOR_" + key + "_at_" + Instant.now();

        }

    }

    static class RedisCache {

        private final Jedis

        jedis; private final int

        ttlSeconds;

        RedisCache(Jedis jedis, int ttlSeconds) {

            this.jedis = jedis;

            this.ttlSeconds = ttlSeconds;

        }

        // Get value: Redis GET; fallback to DB fetch and SET with TTL

        String getOrLoad(String key, SlowDB db) {

            String val =
```

```

jedis.get(key); if (val
!= null) {

    System.out.println("[CACHE HIT] " + key);

    return val;

} else {

    System.out.println("[CACHE MISS] " + key + " -> loading from
    DB"); String fromDb = db.fetch(key);

    // SET key with EX TTL atomically

    SetParams params = new
    SetParams().ex(ttlSeconds); jedis.set(key, fromDb,
    params);

    return fromDb;

}

}

// helper to display approximate memory usage and key
info void printInfo() {

    System.out.println("Redis INFO: " + jedis.info("memory").split("\n")[0]);

    System.out.println("Current TTL of sample key (if exists): " +
jedis.ttl("sample_key"));

}

}

public static void main(String[] args) {

    // Adjust host/port if needed

    try (Jedis jedis = new Jedis("localhost", 6379)) {

        System.out.println("Connected to Redis: " + jedis.ping());

        // NOTE: Configure Redis server separately for LRU eviction:

        // CONFIG SET maxmemory 20mb
        U23CB593 SYSTEM DESIGN MANUAL

        // CONFIG SET maxmemory-policy allkeys-lru

```

```

RedisCache cache = new RedisCache(jedis, 5); // TTL = 5 seconds
SlowDB db = new SlowDB();

// Warm some keys

System.out.println("\n--- Warm cache ---");

cache.getOrLoad("user:1001", db);

cache.getOrLoad("user:1002", db);

// Access pattern: simulate hits & misses

System.out.println("\n--- Access pattern ---");

cache.getOrLoad("user:1001", db); // should be
hit cache.getOrLoad("user:9999", db); // miss

-> load

// Show TTL behaviour

System.out.println("\n--- TTL demonstration (wait for 6 seconds) --
-"); try { Thread.sleep(6000); } catch (InterruptedException
ignored) {} cache.getOrLoad("user:1001", db); // TTL expired ->
miss and reload

// To trigger LRU eviction for demo, you would insert many large keys,
// exceeding maxmemory set in Redis server. Keys least recently used get
evicted. System.out.println("\n--- LRU eviction note ---");

System.out.println("If Redis maxmemory is small, inserting many keys causes
LRU eviction");

System.out.println("Server eviction policy (server-side):
" + jedis.configGet("maxmemory-policy"));

cache.printInfo();

}

}

}

```

Output :

Connected to Redis: PONG

--- Warm cache ---

[CACHE MISS] user:1001 -> loading from

DB [CACHE MISS] user:1002 -> loading
from DB

--- Access pattern ---

[CACHE HIT]

user:1001

[CACHE MISS] user:9999 -> loading from DB

--- TTL demonstration (wait for 6 seconds)

--- [CACHE MISS] user:1001 -> loading
from DB

--- LRU eviction note ---

If Redis maxmemory is small, inserting many keys causes LRU eviction

Server eviction policy (server-side): allkeys-lru

Redis INFO: # Memory

Current TTL of sample key (if exists): -2

Result :

Thus the working of Redis cache for an API by explaining in-memory caching with
TTL (Time-To-Live) and LRU (Least Recently Used) eviction policy was
successfully designed.

Ex.no : 4

PRODUCER-CONSUMER SYSTEM DESIGN USING KAFKA TO DEMONSTRATE AN ORDER PROCESSING QUEUE

Date :

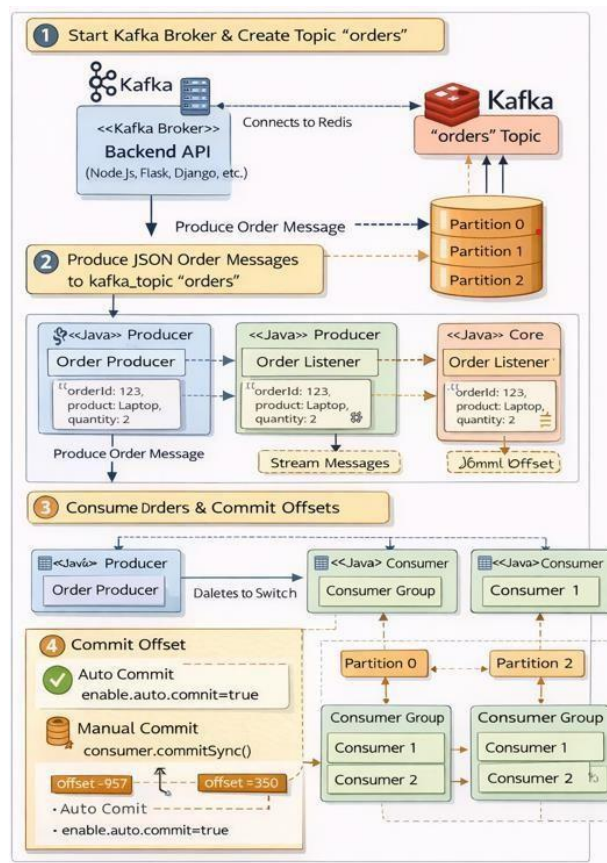
Aim:

To Simulate a producer-consumer system design using Kafka to demonstrate an order processing queue.

Procedure:

1. Start Kafka broker and create topic orders
2. Implement a Java Producer that sends JSON order messages to orders.
3. Implement a Java Consumer that listens to orders, processes each order and commits offsets (either auto or manual commit shown).
4. Simulate multiple producers and multiple consumers (consumer group) to show scaling.

Flow Diagram :



Program:

```
OrderProducer.java

import org.apache.kafka.clients.producer.*;

import

org.apache.kafka.common.serialization.StringSerializer;

import com.fasterxml.jackson.databind.ObjectMapper;

import java.util.Properties;

import java.util.UUID;

public class

OrderProducer {

    private final KafkaProducer<String, String> producer;

    private final String topic;

    private final ObjectMapper mapper = new ObjectMapper();

    public OrderProducer(String bootstrapServers, String topic) {

        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);

        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());

        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());

        // durability settings

        props.put(ProducerConfig.ACKS_CONFIG,

"all");

        props.put(ProducerConfig.RETRIES_CONFIG,

3); this.producer = new

KafkaProducer<>(props); this.topic = topic;

    }

    public void sendOrder(Order order) throws Exception {

        String value = mapper.writeValueAsString(order);
```



```

// key by orderId ensures ordering per key partition
ProducerRecord<String, String> record = new ProducerRecord<>(topic,
order.orderId, value);

producer.send(record, (metadata, exception) -
    > { if (exception != null) {
        System.err.println("Failed to send order " +
            order.orderId); exception.printStackTrace();
    } else {
        System.out.printf("Sent order %s to partition %d offset
            %d%n", order.orderId, metadata.partition(),
            metadata.offset());
    }
});
}

```

```

public void

```

```

    close() {
        producer.flush();
    }

    public void close() {
        producer.close();
    }
}

```

```

public static class

```

```

    Order { public
        String orderId;
        public String
            userId; public
        double amount;

```

```

        public String item;
    }
}

```

```

        public long
        timestamp;

        public Order(String userId, String item, double amount)
        {
            this.orderId = UUID.randomUUID().toString();

            this.userId = userId;

            this.item = item;
            this.amount = amount;

            this.timestamp = System.currentTimeMillis();
        }
    }

    // Simple main to send a few orders

    public static void main(String[] args) throws Exception {
        OrderProducer p = new OrderProducer("localhost:9092",
        "orders"); for (int i = 0; i < 10; i++) {

            Order o = new Order("user-" + (i%3), "item-" + i, 10.5 + i);

            p.sendOrder(o);

            Thread.sleep(200);
        }

        p.close();
    }
}

```

OrderConsumer.java

```

import org.apache.kafka.clients.consumer.*;

import
org.apache.kafka.common.TopicPartition;

import
org.apache.kafka.common.serialization.StringDeserializer;
import com.fasterxml.jackson.databind.ObjectMapper;

```

```

import java.time.Duration;

import java.util.*;

public class OrderConsumer {

    private final KafkaConsumer<String, String> consumer;

    private final String topic;

    private final ObjectMapper mapper = new ObjectMapper();

    public OrderConsumer(String bootstrapServers, String topic, String groupId) {

        Properties props = new Properties();

        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,

            bootstrapServers);

        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,

StringDeserializer.class.getName());

        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,

StringDeserializer.class.getName());

        props.put(ConsumerConfig.GROUP_ID_CONFIG, groupId);

        // disable auto-commit to demonstrate manual commit (at-least-once

behavior)

        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG,

            "false"); props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,

            "earliest");

        this.consumer = new KafkaConsumer<>(props);

        this.topic = topic;

    }

    public void start() {

        consumer.subscribe(Collections.singletonList(topic));

        System.out.println("Consumer started, waiting for

orders..."); try {
            U23CB593 SYSTEM DESIGN MANUAL

            while (true) {

```

```

        ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(1000));

        if (records.isEmpty()) continue;

        for (ConsumerRecord<String, String> rec :

            records) { try {

                OrderProducer.Order order = mapper.readValue(rec.value(),
OrderProducer.Order.class);

                System.out.printf("Processing order %s from user %s amount=%.2f
partition=%d offset=%d%n",

                    order.orderId, order.userId, order.amount, rec.partition(), rec.offset());

            } catch (Exception e) {

System.err.println("Processing failed for record, skipping commit for this batch");
                e.printStackTrace();

            }

        }

// commit offsets after batch processing (manual commit) try {
        consumer.commitSync();
        System.out.println("Offsets committed");

    } catch (CommitFailedException cfe) {

        System.err.println("Commit failed: " + cfe.getMessage());

    }

}

} finally {

    consumer.close();

}

}

public static void main(String[] args) {

    OrderConsumer c = new OrderConsumer("localhost:9092", "orders",
"order-processors");
    c.start();

```

```
}  
}
```

Output :

Sent order 9b2b... to partition 1 offset 12

Processing order 9b2b... from user user-1 amount=11.50 partition=1 offset=12

Offsets committed

Result :

Thus a producer-consumer system design using Kafka to demonstrate an order processing queue was successfully designed.

Ex No : 5	DESIGN AND COMPARE SQL AND NOSQL SCHEMA FOR A BLOGGING PLATFORM, HIGHLIGHTING THE CONCEPTS OF NORMALIZATION AND DENORMALIZATION
Date:	

Aim:

To design and compare SQL and NoSQL schema for a blogging platform, highlighting the concepts of normalization and denormalization.

Procedure:

1. Identify core blog entities: Users, Posts, Comments, Tags.
2. Create relational SQL schema with foreign keys and normalized structure.
3. Create NoSQL denormalized document structure for fast reads.
4. Highlight advantages and disadvantages of both approaches.

Program:

1. SQL Schema (Normalized Design)

Users Table

```
CREATE TABLE Users (
    user_id INT PRIMARY KEY
    AUTO_INCREMENT, username
    VARCHAR(100) UNIQUE NOT NULL, email
    VARCHAR(150) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Posts Table

```
CREATE TABLE Posts (
    post_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT
    NULL, title U23CB593 SYSTEM DESIGN MANUAL
    VARCHAR(255),
```

```
content TEXT,  
  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

Comments Table

```
CREATE TABLE Comments (  
  
    comment_id INT PRIMARY KEY AUTO_INCREMENT,  
  
    post_id INT NOT  
  
    NULL, user_id INT  
  
    NOT NULL,  
  
    comment_text TEXT,  
  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (post_id) REFERENCES Posts(post_id),  
  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

Tags Table

```
CREATE TABLE Tags (  
  
    tag_id INT PRIMARY KEY AUTO_INCREMENT,  
  
    tag_name VARCHAR(50) UNIQUE  
  
);
```

Post_Tags Mapping Table (Many-to-Many)

```
CREATE TABLE Post_Tags (  
  
    post_id INT,  
  
    tag_id INT,  
  
    PRIMARY KEY (post_id, tag_id)  
);
```

2. Sample SQL Insert Operations :

```
INSERT INTO Users (username, email)
VALUES ('john_doe', 'john@gmail.com');
```

```
INSERT INTO Posts (user_id, title, content)
VALUES (1, 'My First Blog', 'This is a blog about databases');
```

```
INSERT INTO Tags
(tag_name) VALUES
('database'), ('sql');
```

```
INSERT INTO Post_Tags (post_id,
tag_id) VALUES (1, 1), (1, 2);
```

```
INSERT INTO Comments (post_id, user_id, comment_text)
VALUES (1, 1, 'Great post!');
```

3. SQL Read Query (Requires

Joins) SELECT u.username, p.title,
c.comment_text FROM Users u
JOIN Posts p ON u.user_id = p.user_id
JOIN Comments c ON p.post_id =
c.post_id WHERE p.post_id = 1;

NoSQL (Document Database – MongoDB)

1. Posts Collection (Denormalized Design)

```
{
  "_id":
    ObjectId("65afc12a9b1"),
  "user": {
    "user_id": 1,
    "username":
      "john_doe",
    "email": "john@gmail.com"
```



```

    },
    "title": "My First Blog",
    "content": "This is a blog about
databases", "tags": ["database", "nosql"],
    "comments": [
        {
            "user_id": 2,
            "username": "alice",
            "comment_text": "Very informative!",
            "created_at": "2025-01-02T10:30:00"
        }
    ],
    "created_at": "2025-01-02T09:00:00"
}

```

2. MongoDB Insert Operation

db.posts.insertOne

```

({ user: {
    user_id: 1,
    username: "john_doe",
    email:
        "john@gmail.com"
    },
    title: "My First Blog",
    content: "This is a blog about databases",
    tags: ["database", "nosql"],
    comments: [
        {
            user_id: 2,
            username:
                "alice",
            comment_text: "Very informative!",
            created_at: new Date()
        }
    ],
    created_at: new Date()
}

```

```
});
```

3. MongoDB Read Query (No Joins Needed)

```
db.posts.find(  
  { title: "My First Blog" },  
  { title: 1, comments: 1 }  
);
```

Result :

Thus the SQL and NoSQL schema for a blogging platform, highlighting the concepts of normalization and denormalization was successfully designed.

Ex No : 6	IMPLEMENTATION OF CIRCUIT BREAKER AND RETRY LOGIC USING RESILIENCE4J
Date:	

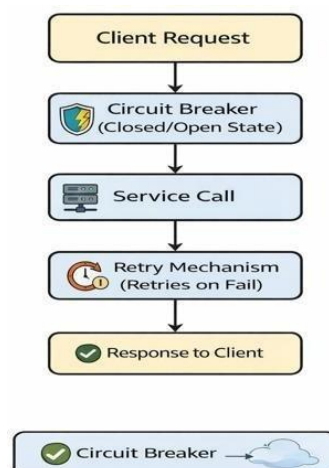
Aim:

To implement and demonstrate Circuit Breaker and Retry logic in a microservice using Resilience4j to handle service failures gracefully and improve fault tolerance.

Procedure:

1. Define the purpose of the Circuit Breaker — to prevent repeated calls to a failing service and protect the system from cascading failures.
2. Configure the Circuit Breaker with parameters like failure rate threshold, wait duration in open state, and sliding window size.
3. Define the Retry logic configuration with parameters such as maximum retry attempts and waiting duration between retries.
4. Create a simulated remote service call that may fail randomly to test the fault-tolerance behavior.
5. Wrap the service call using Resilience4j decorators that combine both Circuit Breaker and Retry logic.
6. Execute multiple service requests and observe how the Circuit Breaker opens after consecutive failures and retries occur automatically.
7. Verify that after the wait duration, the Circuit Breaker moves to the half-open state, allowing limited test requests before closing again upon success.

Diagram:



Program :

```
// ServiceCaller.java
```

```
import io.github.resilience4j.circuitbreaker.*;
import io.github.resilience4j.retry.*;
import io.github.resilience4j.decorators.Decorators;
import java.util.function.Supplier;

public class ServiceCaller {

    public static void main(String[] args) throws Exception
    {

        // Create CircuitBreaker configuration
        CircuitBreakerConfig circuitBreakerConfig = CircuitBreakerConfig.custom()
            .failureRateThreshold(50)
            .waitDurationInOpenState(java.time.Duration.ofSeconds(5))
            .slidingWindowSize(5)
            .build();

        CircuitBreaker circuitBreaker = CircuitBreaker.of("myCircuitBreaker",
circuitBreakerConfig);

        // Create Retry configuration
        RetryConfig retryConfig = RetryConfig.custom()
            .maxAttempts(3)
            .waitDuration(java.time.Duration.ofSeconds(1))
            .build();

        Retry retry = Retry.of("myRetry", retryConfig)

        // Simulated remote service call
        Supplier<String> remoteService = () -> {

            System.out.println("Attempting remote service
call..."); if (Math.random() < 0.7) { // 70% chance of
failure

                throw new RuntimeException("Service failed!");
            }

            return "Success Response from Service";
        }
    }
}
```

```

};

// Combine CircuitBreaker and Retry
Supplier<String> decoratedSupplier = Decorators.ofSupplier(remoteService)
    .withCircuitBreaker(circuitBreaker)
    .withRetry(retry)
    .decorate();
for (int i = 1; i <= 10; i++)
{ try {
    String result = decoratedSupplier.get();
    System.out.println("Request " + i + ": " + result);
} catch (Exception e) {
    System.out.println("Request " + i + ": " + e.getMessage());
}
    Thread.sleep(1000);
}
}
}

```

Output :

Attempting remote service

call... Request 1: Service

failed!

Attempting remote service

call... Request 2: Service

failed!

Attempting remote service

call... Request 3: Service

failed!

CircuitBreaker is now OPEN - blocking further calls

U23CB593 SYSTEM DESIGN MANUAL

Request 4: CircuitBreaker 'myCircuitBreaker' is OPEN and does not permit further

calls Request 5: CircuitBreaker 'myCircuitBreaker' is OPEN and does not permit further calls

(After 5 seconds)

Attempting remote service call...

Request 6: Success Response from

Service CircuitBreaker is now

CLOSED again Request 7: Success

Response from Service

Result :

Thus, the Circuit Breaker and Retry mechanisms were successfully implemented using Resilience4j in Java to improve system resilience and handle transient failures gracefully in microservice architectures.

Ex No : 7	WORKING OF TOKEN BUCKET RATE LIMITER
Date:	

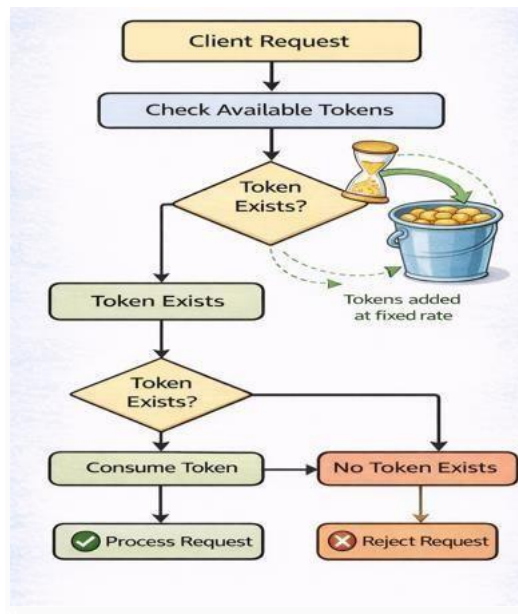
Aim:

To implement and demonstrate a Token Bucket Rate Limiter in Java to control the rate of incoming API requests.

Procedure:

1. Initialize a bucket with a fixed capacity and token refill rate.
2. Periodically add tokens to the bucket without exceeding capacity.
3. When a client request arrives:
 - Check token availability.
 - If tokens are available, allow the request and deduct tokens.
 - If not, reject the request.
4. Repeat the process continuously for all incoming requests.
5. Observe how bursts are allowed while maintaining rate control.

Flow Diagram :



Program :

```

import
java.time.Instant;
public class
TokenBucket {

```

```

private final int capacity;      // Maximum
tokens private final int refillRate;      //
Tokens per second private double tokens;
                                   // Current token
count private long lastRefillTimestamp; // Last
refill time

public TokenBucket(int capacity, int refillRate) {
    this.capacity = capacity;
    this.refillRate =
    refillRate; this.tokens =
    capacity;
    this.lastRefillTimestamp = Instant.now().getEpochSecond();
}

// Refill tokens based on elapsed time
private void refill() {
    long currentTime = Instant.now().getEpochSecond();
    long elapsedTime = currentTime -
    lastRefillTimestamp;
    if (elapsedTime > 0) {
        double tokensToAdd = elapsedTime * refillRate;
        tokens = Math.min(capacity, tokens +
        tokensToAdd); lastRefillTimestamp =
        currentTime;
    }
}
// Check whether request is allowed
public synchronized boolean allowRequest()
{ refill();
  if (tokens >=
    1) {
        tokens--;
        return
        true;

```



```

    }
    return false;
}
}

Main.java
public class Main {
    public static void main(String[] args) throws InterruptedException {
        // Bucket capacity = 5 tokens, refill rate = 1
        token/sec TokenBucket bucket = new
        TokenBucket(5, 1);
        for (int i = 1; i <= 10; i++) {
            if (bucket.allowRequest()) {
                System.out.println("Request " + i + ":
                Allowed");
            } else {
                System.out.println("Request " + i + ": Rejected");
            }
            Thread.sleep(500); // Simulate request interval
        }
    }
}

```

Output:

```

Request 1: Allowed
Request 2: Allowed
Request 3: Allowed
Request 4: Allowed
Request 5: Allowed
Request 6: Rejected
Request 7: Allowed
Request 8: Rejected

```

Result:

The Token Bucket Rate Limiter was successfully implemented in Java. The program demonstrated controlled request handling by allowing requests only when tokens were available.

Ex No : 8	FAILOVER AND HEARTBEAT MECHANISMS IN A LOAD-BALANCED APPLICATION USING MOCK SERVICES
Date:	

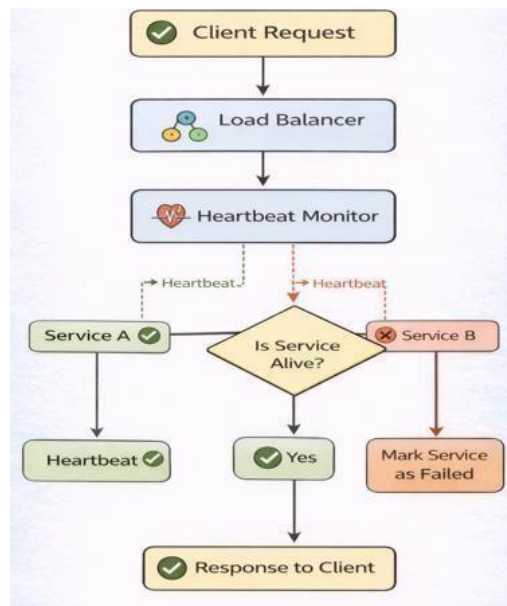
Aim

To study and implement failover and heartbeat mechanisms in a load-balanced application using mock services, ensuring high availability by detecting service failures and rerouting traffic to healthy instances.

Procedure

1. Create multiple mock backend services (Service A, Service B).
2. Implement a heartbeat mechanism where each service periodically reports its health.
3. Develop a Load Balancer that:
 - Maintains a list of services
 - Checks heartbeat status
4. Route client requests only to healthy services.
5. Simulate a service failure by stopping heartbeat updates.
6. Observe failover, where traffic is redirected to available services.
7. Restore the failed service and observe it rejoining the system.

Flow Diagram :



Program :

LoadBalancer.java

```
import java.util.List;
```

```

public class LoadBalancer {
    private List<ServiceInstance> services;
    private long heartbeatTimeout = 3000; // 3 seconds
    public LoadBalancer(List<ServiceInstance> services)
    {
        this.services = services;
    }

    public ServiceInstance routeRequest()
    {
        for (ServiceInstance service :
            services) {
            if (service.isAlive(heartbeatTimeout)) {
                return service;
            }
        }
        return null;
    }
}

```

Main.java

(Simulation) import

java.util.Arrays; public

class Main {

public static void main(String[] args) throws InterruptedException {

ServiceInstance serviceA = new ServiceInstance("Service-A");

ServiceInstance serviceB = new ServiceInstance("Service-B");

LoadBalancer lb = new LoadBalancer(Arrays.asList(serviceA,

serviceB)); for (int i = 1; i <= 10; i++) {

 // Simulate

 heartbeat if (i <=

 5) {

 serviceA.sendHeartbeat(); // Service A alive initially

 }

 serviceB.sendHeartbeat();

 ServiceInstance service =

 lb.routeRequest(); if (service != null) {

```

        System.out.println("Request " + i + " routed to " + service.getServiceName());
    } else {
        System.out.println("Request " + i + " failed: No service available");
    }
    Thread.sleep(1000);
}
}
}

```

Output:

```

Request 1 routed to
Service-A Request 2
routed to Service-A
Request 3 routed to
Service-A Request 4
routed to Service-B
Request 5 routed to
Service-B Request 6
routed to Service-B

```

Result:

Failover and heartbeat mechanisms were successfully implemented using mock services. The system automatically detected service failures and rerouted requests to healthy instances, ensuring uninterrupted service.

Ex No: 9	DESIGN OF MICROSERVICES FOR E-COMMERCE CHECKOUT SYSTEM USING REST API
Date:	

Aim

To design and implement a microservices-based e-commerce checkout system using REST API concepts by separating responsibilities into Cart, Inventory, and Payment services.

Procedure

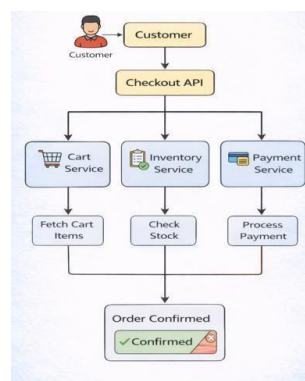
1. Design individual microservices with single responsibility.
2. Expose REST APIs using HTTP methods (GET, POST).
3. Cart Service stores selected products.
4. Inventory Service validates stock availability.
5. Payment Service processes payment.
6. Checkout service coordinates calls in sequence.
7. If any step fails, checkout is aborted.

Microservices Identified

1. Cart Service – Manages user cart items
2. Inventory Service – Checks and updates product stock
3. Payment Service – Processes payment
4. Checkout Orchestrator (Client / API Gateway) – Coordinates all services

Flow Diagram:

Microservices for E-Commerce Checkout System using REST API



Program :**CartService.java**

```
import java.util.*;

public class CartService {

    public List<String> getCartItems(String
        userId) { return Arrays.asList("Laptop",
        "Mouse");
    }
}
```

InventoryService.java

```
import java.util.*;

public class InventoryService {

    private Map<String, Integer> stock = new
    HashMap<>(); public InventoryService() {
        stock.put("Laptop", 5);
        stock.put("Mouse", 10);
    }

    public boolean checkStock(List<String>
        items) { for (String item : items) {
        if (!stock.containsKey(item) || stock.get(item) <= 0) {
            return false;
        }
    }
    return true;
}
}
```

PaymentService.java

```
public class PaymentService {

    public boolean processPayment(String userId, double amount)
    { System.out.println("Payment processed for user " +
        userId); return true;
    }
}
```

```
}
```

CheckoutService.java

```
import java.util.List;
```

```
public class CheckoutService {
```

```
    private CartService cartService = new CartService();
```

```
    private InventoryService inventoryService = new InventoryService();
```

```
    private PaymentService paymentService = new PaymentService();
```

```
    public void checkout(String userId) {
```

```
        List<String> cartItems =
```

```
        cartService.getCartItems(userId); if
```

```
        (!inventoryService.checkStock(cartItems)) {
```

```
            System.out.println("Checkout Failed: Stock
```

```
            unavailable"); return;
```

```
        }
```

```
        boolean paymentStatus = paymentService.processPayment(userId,
```

```
        50000); if (paymentStatus) {
```

```
            System.out.println("Order placed successfully!");
```

```
        } else {
```

```
            System.out.println("Payment failed");
```

```
        }
```

```
    }
```

```
}
```

Main.java

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        CheckoutService checkoutService = new
```

```
        CheckoutService();
```

```
        checkoutService.checkout("USER123");
```

```
    }
```

```
}
```

Output:

Payment processed for user
USER123 Order placed
successfully!

Result:

The microservices-based e-commerce checkout system was successfully designed and implemented using REST API concepts. The system demonstrated modular design, service interaction, and a real-world checkout workflow.

U23CB593 SYSTEM DESIGN MANUAL

Ex No: 10	IMPLEMENTATION OF PUB/SUB NOTIFICATION SERVICE USING RABBITMQ
Date:	

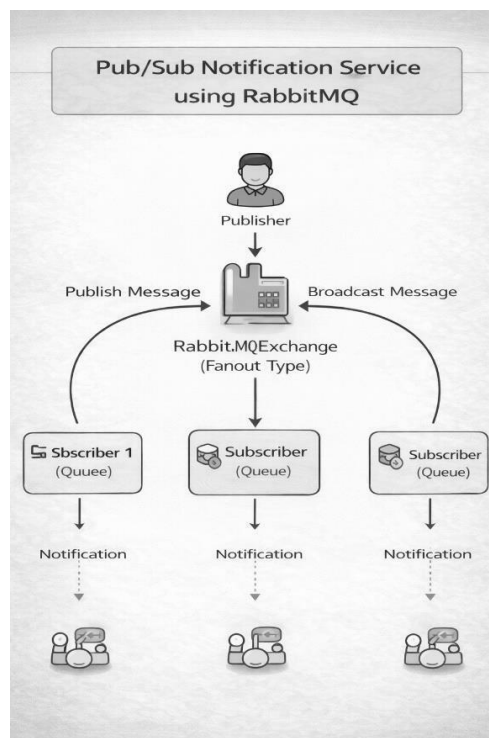
Aim

To design and implement a Publish–Subscribe (Pub/Sub) notification service using RabbitMQ, where publishers send messages to an exchange and multiple subscribers receive notifications asynchronously.

Procedure

1. Install and start RabbitMQ Server.
2. Create a Fanout Exchange for broadcasting messages.
3. Implement a Publisher (Producer) to send notification messages.
4. Implement multiple Subscribers (Consumers) that listen to the exchange.
5. Bind each subscriber queue to the exchange.
6. Publish messages and observe delivery to all subscribers.

Flow Diagram:



Program:**Publisher.java**

```
import com.rabbitmq.client.Channel;
import
com.rabbitmq.client.Connection;
import com.rabbitmq.client.ConnectionFactory;
public class Publisher {
    private static final String EXCHANGE_NAME = "notifications_exchange";
    public static void main(String[] args) throws Exception {
        ConnectionFactory factory = new
        ConnectionFactory(); factory.setHost("localhost");
        Connection connection = factory.newConnection();
        Channel channel = connection.createChannel();
        channel.exchangeDeclare(EXCHANGE_NAME,
        "fanout"); String message = "New Order Placed!";
        channel.basicPublish(EXCHANGE_NAME, "", null, message.getBytes());
        System.out.println("Published Message: " + message);
        channel.close();
        connection.close();
    }
}
```

Subscriber.java

```
import com.rabbitmq.client.*;
public class Subscriber {
    private static final String EXCHANGE_NAME = "notifications_exchange";
    public static void main(String[] args) throws Exception {
        ConnectionFactory factory = new
        ConnectionFactory(); factory.setHost("localhost");
        Connection connection = factory.newConnection();
        Channel channel = connection.createChannel();
        channel.exchangeDeclare(EXCHANGE_NAME,
        "fanout"); String queueName =
        channel.queueDeclare().getQueue();
        channel.queueBind(queueName, EXCHANGE_NAME,
        "");
    }
}
```

```
System.out.println("Waiting for notifications...");
DeliverCallback callback = (consumerTag, delivery) -> {
    String message = new String(delivery.getBody(), "UTF-8");
    System.out.println("Received Notification: " + message);
};

channel.basicConsume(queueName, true, callback, consumerTag -> {});
}
}
```

Output:

Publisher

Published Message: New Order Placed!

Subscriber 1

Received Notification: New Order

Placed! Subscriber 2

Received Notification: New Order Placed!

Result

The Pub/Sub Notification Service was successfully implemented using RabbitMQ. Messages published by the producer were delivered to all subscribers, demonstrating asynchronous communication and event-driven architecture.

Ex No : 11	DEMONSTRATION OF API GATEWAY ARCHITECTURE USING EXPRESS.JS OR NGINX
Date:	

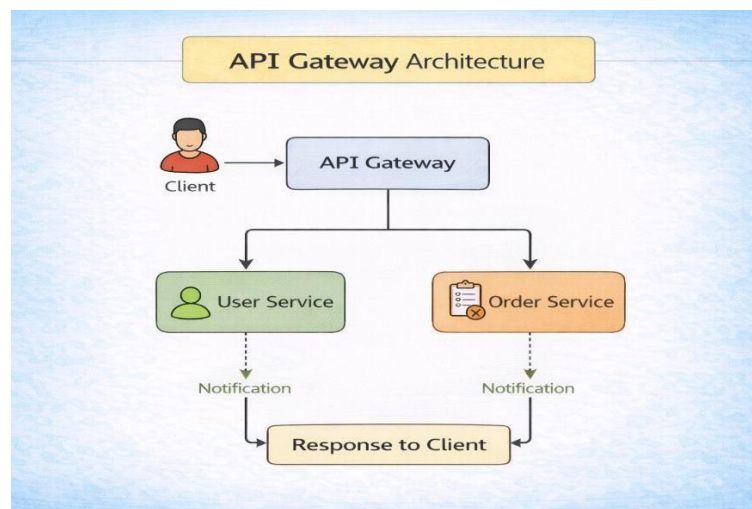
Aim

To demonstrate the working of an API Gateway architecture that acts as a single entry point for multiple backend microservices, providing request routing, abstraction, and centralized control.

Procedure

1. Identify backend services (User Service, Order Service).
2. Design an API Gateway that receives all client requests.
3. Based on the request path, route requests to appropriate services.
4. Each service processes the request independently.
5. API Gateway aggregates or forwards responses back to the client.
6. Observe that the client interacts only with the gateway.

Flow Diagram :



Program:

UserService.java

```

public class OrderService {
    public String
    handleRequest() {
        return "Order Service Response";
    }
}

```

OrderService.java

```
public class OrderService {  
    public String  
    handleRequest() {  
        return "Order Service Response";  
    }  
}
```

APIGateway.java

```
public class APIGateway {  
    private UserService userService = new  
    UserService(); private OrderService orderService =  
    new OrderService(); public String  
    routeRequest(String path) {  
        if (path.equals("/users")) {  
            return userService.handleRequest();  
        } else if (path.equals("/orders")) {  
            return  
            orderService.handleRequest();  
        } else {  
            return "404 - Service Not Found";  
        }  
    }  
}
```

Main.java

```
public class Main {  
    public static void main(String[] args) {  
        APIGateway gateway = new  
        APIGateway();  
        System.out.println("Client Request:  
        /users");  
        System.out.println("Response: " +  
        gateway.routeRequest("/users")); System.out.println("\nClient  
        Request: /orders"); System.out.println("Response: " +  
        gateway.routeRequest("/orders")); U23CB593 SYSTEM DESIGN MANUAL  
    }  
}
```

}

Output:

Client Request: /users

Response: User Service Response

Client Request: /orders

Response: Order Service Response

Result

The API Gateway architecture was successfully demonstrated using a Java-based simulation. The system showed how a single gateway routes requests to multiple backend services, simplifying client communication.

Ex No : 12

Date:

ARCHITECTURE DIAGRAM FOR REAL-TIME CHAT APPLICATION

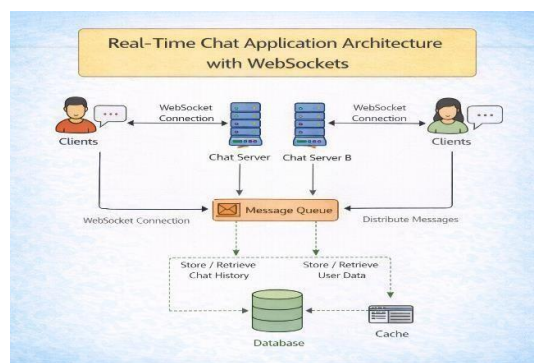
Aim

To design and understand the architecture of a real-time chat application that supports scalability using WebSockets, message queues, caching, and databases for efficient and reliable message delivery.

Procedure

1. Identify the system components required for a real-time chat application such as clients, chat servers, message queue, cache, and database.
2. Establish persistent WebSocket connections between clients and chat servers to enable real-time, bi-directional communication.
3. Deploy multiple chat servers to support horizontal scaling and handle a large number of concurrent users.
4. Integrate a message queue to distribute messages across different chat servers when sender and receiver are connected to different servers.
5. Store chat messages and user-related information in a database for persistence and future retrieval.
6. Use a cache to store frequently accessed data such as recent messages and online user status to improve performance.
7. Route messages from clients through chat servers and the message queue to ensure reliable and scalable message delivery.
8. Verify that the system continues to function efficiently as the number of users increases.

Architecture Diagram :



The diagram represents a scalable real-time chat application designed using WebSockets, message queues, cache, and databases to support a large number of concurrent users.

1. Clients

- Users connect to the system using web or mobile applications.
- A persistent WebSocket connection is established with the chat server.
- This allows real-time, bi-directional communication.

2. WebSocket Connections

- WebSockets maintain an always-open connection between clients and chat servers.
- Messages are delivered instantly without repeated HTTP requests.
- Suitable for live chats, typing indicators, and presence updates.

3. Chat Servers (A & B)

- Multiple chat servers run in parallel to support horizontal scaling.
- Each server handles a subset of connected users.
- Load balancers can distribute users across these servers.
- Servers are stateless, relying on shared messaging infrastructure.

4. Message Queue

- A message queue (e.g., Kafka, RabbitMQ) ensures:
 - Reliable message delivery
 - Decoupling between chat servers
 - Support for cross-server communication
- Messages sent from one server are published to the queue and delivered to other servers.

5. Message Distribution

- If sender and receiver are connected to different chat servers, the queue distributes the message correctly.
- This ensures scalability and fault tolerance.

6. Database

- The database stores:
 - Chat history
 - User profiles
 - Conversation metadata
- Supports message persistence and retrieval when users reconnect.

7. Cache

- Cache (e.g., Redis) stores:
 - Recent messages
 - Online/offline user status
 - Frequently accessed data
- Improves performance by reducing database load.

8. Store and Retrieve Flow

- Messages flow from clients → chat servers → message queue.
- Messages are stored in the database.
- Cached data is used for faster access where applicable.

Scalability Features Highlighted

- Horizontal scaling using multiple chat servers
- Asynchronous communication via message queues
- Reduced latency using cache
- High availability using distributed components

Result:

This architecture efficiently supports real-time messaging at scale by combining WebSockets for instant communication, message queues for reliable distribution, caching for performance, and databases for persistence.

Ex No : 13	IMPLEMENTATION OF SERVLET-BASED USER MANAGEMENT WITH CRUD OPERATIONS.
Date:	

Aim:

To develop a Servlet-based User Management Application that performs CRUD (Create, Read, Update, Delete) operations on user data using Java Servlets, JSP, and JDBC.

Procedure:

1. Create a **database** userdb with table users(id, name, email, country).
2. Create a **Java Servlet project** and configure **JDBC** connection.
3. Create:
 - a. User.java – Model class
 - b. UserDAO.java – Handles DB CRUD operations
 - c. UserServicelet.java – Controls request/response
4. Create JSP pages – user-form.jsp and user-list.jsp.
5. Configure servlet in web.xml and run on Tomcat.

Program :**// User.java**

```
package
model; public
class User {
    private int id;
    private String
    name; private
    String email;
    private String country;

    public User(int id, String name, String email, String
    country) { this.id = id;
    this.name = name;
    this.email = email;
    this.country =
    country;
    }

    public User(String name, String email, String country) {
        this.name = name; U23CB593 SYSTEM DESIGN MANUAL
```

```

        this.email = email;
        this.country =
        country;
    }

    // Getters and Setters
    public int getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getCountry() { return
    country; }

    public void setId(int id) { this.id = id; }
    public void setName(String name) { this.name =
    name; } public void setEmail(String email) {
    this.email = email; }
    public void setCountry(String country) { this.country = country; }
    }

//UserDAO.java
    import java.sql.*; import java.util.*; public class UserDAO
    { Connection getCon() throws Exception { return
    DriverManager.getConnection("jdbc:mysql://localhost:3306/userdb","root","password")
    ;
    }
    public void insert(User u)throws Exception{ PreparedStatement
    ps=getCon().prepareStatement("insert into
    users(name,email,country)values(?,?,?)");
    ps.setString(1,u.name);ps.setString(2,u.email);ps.setString(3,u.country);ps.executeUpdate ();
    }
    public List<User> getAll()throws Exception{ List<User> list=new ArrayList<>();
    ResultSet rs=getCon().createStatement().executeQuery("select * from users");
    while(rs.next())list.add(new
    User(rs.getInt(1),rs.getString(2),rs.getString(3),rs.getString(4)));
    return list;
    }
    }

```

```
//UserServlet.java
@WebServlet("/")

public class UserServlet extends HttpServlet { UserDao dao=new UserDao();
protected void doGet(HttpServletRequest r,HttpServletResponse s)throws
IOException,ServletException{
try{
List<User> list=dao.getAll(); r.setAttribute("list",list); r.getRequestDispatcher("user-
list.jsp").forward(r,s);
} catch(Exception e){e.printStackTrace();}
}

protected void doPost(HttpServletRequest r,HttpServletResponse s)throws
IOException{ try {dao.insert(new
User(r.getParameter("name"),r.getParameter("email"),r.getParameter("country")));
s.sendRedirect("list");} catch(Exception e){e.printStackTrace();}
}
}
```

Output :

1. User List Page

ID	Name	Email	Country	Actions
1	John	john@gmail.com	India	Edit Delete
2	Alice	alice@yahoo.com	USA	Edit Delete

2. Add / Edit Form

Name:

Email: Country: [Save] [Cancel]

Result :

Thus, a Servlet-based User Management Application was successfully developed and implemented in Java using JSP, Servlets, and JDBC to perform and manage CRUD operations on user data efficiently.

Ex No : 14	REST API ARCHITECTURE USING SPRING BOOT OR JAX-RS FOR A TASK MANAGEMENT SYSTEM.
Date:	

Aim:

To design and implement a RESTful API architecture for a Task Management System using Spring Boot.

Procedure:

1. Define models: Task, User, Project.
2. Use Spring Boot with JPA for data persistence.
3. Implement Controller → Service → Repository layers.
4. Provide CRUD operations for tasks.
5. Add a Token Bucket Rate Limiter filter for API request control.

Program :

```
//Entity
import javax.persistence.*;

@Entity
public class Task {

    @Id
    @GeneratedValue
    private Long id;

    private String title;

    private String
    description; private
    String status;

}
//Repository
public interface TaskRepository extends JpaRepository<Task, Long> {}

//Service
```

```

@Service

public class TaskService {

    @Autowired private TaskRepository repo;
    public List<Task> getAll() { return

        repo.findAll(); } public Task create(Task t) {

        return repo.save(t); }

}

//Controller

@RestController

@RequestMapping("/api/tas

ks") public class

TaskController {

    @Autowired private TaskService service;
    @GetMapping public List<Task> getAll() { return service.getAll(); }
    @PostMapping public Task create(@RequestBody Task t) { return service.create(t); }

}

//Rate Limiter Filter

@Component

public class

TokenBucketFilter

extends

OncePerRequestFilter {

    private int tokens = 5; //

    capacity

    @Override

    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,

    FilterChain chain)throws IOException, ServletException {

```

```
    if (tokens > 0) { tokens--; chain.doFilter(req, res); }  
    else { res.setStatus(429); res.getWriter().write("Rate limit exceeded"); }  
  }  
}
```

Output :

GET /api/tasks [

```
{ "id": 1, "title": "Design API", "description": "Prepare task endpoints", "status": "TODO" }
```

]

POST /api/tasks

```
{ "id": 2, "title": "Implement service", "description": "Add business logic", "status": "IN_PROGRESS" }
```

Result :

Thus, A simple REST API for task management is successfully designed and implemented using Spring Boot, supporting CRUD operations and protected by a Token Bucket Rate Limiter.

Ex No : 15	PREPARE A CAPSTONE SYSTEM DESIGN FOR A TWITTER-LIKE PLATFORM FOCUSING ON DATABASE SHARDING, FEED GENERATION, AND CACHING STRATEGIES
Date:	

Aim

To design and implement a Twitter-like microblogging platform with scalable database sharding, efficient feed generation, and caching strategies to support millions of users with real-time updates.

Objectives

- Implement user, tweet, and follow relationships in a scalable way
- Generate personalized feeds efficiently for each user
- Use database sharding to distribute data across multiple servers
- Apply caching strategies (Redis / Memcached) for faster feed retrieval
- Ensure high availability and fault tolerance

System Requirements

Functional

Requirements

- Users can post tweets
- Users can follow/unfollow others
- Users can view their personalized feed
- Users can like/retweet posts

Non-Functional Requirements

- Support millions of users and tweets
- Low-latency feed retrieval (<500ms)
- Scalable storage for tweets
- High availability (no single point of failure)

High-Level Design

(HLD) Components

- Web / Mobile Clients – Send requests to API
- API Gateway – Routes requests to services
- User Service – Manages user data and follow relationships
- Tweet Service – Handles tweet creation and deletion
- Feed Service – Generates and serves feeds
- Database Layer – Stores users, tweets, and follow data
 - Sharded by user_id or tweet_id
- Cache Layer (Redis / Memcached) – Stores precomputed feeds and popular tweets
- Message Queue (Kafka / RabbitMQ) – Asynchronous feed updates

Database Sharding Strategy

- Users Table: Sharded by user_id
- Tweets Table: Sharded by user_id or tweet_id
- Follow Table: Sharded by follower_id

Feed Generation Strategies

Pull Model – Generate feed on request

- Pros: Real-time, simple
- Cons: Slow for users following many accounts

Push Model – Precompute feeds and store in cache

- Pros: Fast feed retrieval
- Cons: More storage required

Caching Strategies

- Cache latest tweets for popular users
- Cache precomputed feeds for active users
- Set TTL for cache eviction
- Use LRU eviction policy for limited memory

Low-Level Design (LLD)

Tables / Collections:

Users Table

Column	Type	Description
user_id	BIGINT	Primary Key
username	VARCHAR	Unique username
email	VARCHAR	Unique
created_at	TIMESTAMP	Registration time

Tweets Table

Column	Type	Description
tweet_id	BIGINT	Primary Key
user_id	BIGINT	Foreign Key
content	TEXT	Tweet content
created_at	TIMESTAMP	Timestamp

Follows Table

Column	Type	Description
follower_id	BIGINT	User who follows
followee_id	BIGINT	User being followed
created_at	TIMESTAMP	Timestamp

Cache

- Key: feed:user_id → Value: list of tweet_ids
- Key: tweet:tweet_id → Value: tweet content

Message Queue Usage

- When a user posts a tweet:
 1. Tweet saved in database shard
 2. Feed service receives event via queue
 3. Feeds of followers updated asynchronously in cache

Tweet Event Producer

```
public class TweetService {  
    private MessageQueue queue; // Kafka/RabbitMQ  
    public void postTweet(long userId, String content) {  
        Tweet tweet = new Tweet(userId, content);  
        database.save(tweet); // Save in DB shard  
        queue.publish("feed-update", tweet); // Async feed update  
    }  
}
```

Feed Consumer

```
public class FeedService {  
    private Cache redis;  
    public void consumeTweet(Tweet tweet) {  
        List<Long> followers =  
            database.getFollowers(tweet.getUserId());  
        for (Long followerId : followers) {  
            redis.prependToFeed(followerId, tweet.getTweetId());  
        }  
    }  
}
```

Feed Retrieval

```
public List<Tweet> getFeed(long userId) {  
    List<Long> tweetIds = redis.getFeed(userId);  
    if (tweetIds.isEmpty()) {  
        tweetIds =  
            database.fetchRecentTweets(userId);  
        redis.setFeed(userId, tweetIds);  
    }  
    return database.fetchTweets(tweetIds);  
}
```

Result:

The Twitter-like system design demonstrates how database sharding, caching, and asynchronous feed generation work together to build a scalable, high-performance social media platform.